

Implementação de um Sistema de Arquivos Hierárquico Baseado em Inodes (TreeFS) em um Microkernel RISC-V

Isadora de Mello, Juliana Bertolazi Simon, Nicolas Vera

Disciplina de Sistemas Operacionais – Curso de Ciência da Computação – UNIVALI

Julho de 2026

1. Enunciado do Trabalho

A avaliação M3 de Sistemas Operacionais pede a evolução do microkernel que vínhamos desenvolvendo ao longo do semestre, o BasicMicrokernel, para que ele passasse a ter um sistema de arquivos hierárquico, batizado de TreeFS pelo professor. A proposta é inspirada em sistemas Unix reais, como o Minix, o EXT2 e o XV6, mas simplificada para caber no contexto da disciplina.

No mínimo, o trabalho precisava entregar organização hierárquica de diretórios, uso de inodes para guardar metadados, separação clara entre metadados e dados, gerenciamento de blocos e de inodes livres por bitmap, resolução de caminhos absolutos através de uma função `path_lookup`, criação dinâmica de arquivos e diretórios, e a integração de tudo isso com o restante do kernel que já tínhamos pronto da M2, ou seja, a gerência de memória e o escalonador.

Como referência, o enunciado aponta o Capítulo 17 do manual da disciplina, que trata da implementação de um sistema de arquivos hierárquico baseado em inodes. Esse capítulo descreve a arquitetura em camadas usada como base (aplicações, API de arquivos, TreeFS, driver de blocos e disco virtual), as estruturas de dados esperadas (superblock, inode e entrada de diretório) e o fluxo esperado das principais operações, como `fs_init`, `path_lookup`, `mkdir`, `create`, `write`, `read`, `unlink` e `ls`.

Além do que era obrigatório, o enunciado sugeria algumas funcionalidades extras para quem quisesse nota complementar: timestamps, permissões, links simbólicos, links físicos, blocos indiretos, cache de blocos e suporte a múltiplos usuários. Explicamos na Seção 3.8 quais desses itens conseguimos implementar e por que decidimos deixar os outros de fora.

2. Introdução e Contexto

Um sistema de arquivos existe para organizar, guardar e recuperar informação de forma persistente, e é uma das partes mais importantes de qualquer sistema

operacional. Nos sistemas Unix, essa organização acontece através de uma estrutura hierárquica de diretórios, apoiada no conceito de inode: uma estrutura de metadados que não guarda o nome do arquivo, esse nome fica separado, dentro das entradas de diretório.

Este trabalho continua o projeto que começamos na M2, onde já tínhamos entregado um gerenciador de memória com lista livre, usando a política First Fit, com split e coalescência de blocos, e um escalonador preemptivo Round-Robin baseado em interrupções de timer. O TreeFS foi construído em cima dessa base, usando o mesmo `kmalloc()/kfree()` que já estava validado para reservar as áreas do disco virtual.

No final, o sistema consegue formatar um disco virtual em memória, navegar pela árvore de diretórios e criar, ler, escrever e remover arquivos usando caminhos absolutos. Em escala bem menor, isso reproduz os mesmos mecanismos internos de um sistema de arquivos real, como o EXT2.

3. Desenvolvimento

3.1 Arquitetura do TreeFS

Seguimos a arquitetura em camadas proposta no material de referência da disciplina: as aplicações e tasks chamam a API de arquivos (`fs_init`, `mkdir`, `create`, `ls`, `unlink`, `read`, `write`), que é implementada pelo TreeFS, responsável pelos inodes, blocos e diretórios, que por sua vez usaria um driver de blocos para acessar o disco virtual.

Precisamos adaptar um ponto da proposta original. A placa virt do QEMU que usamos não expõe nenhum dispositivo de bloco físico acessível pelos drivers que já tínhamos das etapas anteriores. Por isso, o papel do disco virtual e do driver de blocos acabou sendo feito por uma região contígua de memória RAM, reservada com `kmalloc()` dentro de `fs_init()`. É a mesma estratégia sugerida pelo próprio material de referência, chamada lá de disco virtual na memória RAM, e está dentro do que o enunciado permite, já que ele autoriza adaptar a organização desde que isso seja justificado.

3.2 Superblock e Bitmaps de Alocação

O superblock (`superblock_t`) guarda a assinatura do sistema de arquivos (`magic` = 0x54524545, que corresponde a “TREE” em hexadecimal), o total de blocos (256),

o total de inodes (128) e o tamanho de cada bloco (512 bytes). Com esses números, a área de dados do disco virtual acaba tendo 256 vezes 512, ou seja, 131072 bytes, os mesmos 128 KiB.

Inodes e blocos livres são controlados por dois bitmaps, `inode_bitmap` e `block_bitmap`, em que cada bit diz se aquela posição está livre (0) ou ocupada (1). As funções `inode_alloc()/inode_free()` e `block_alloc()/block_free()` percorrem esses bitmaps procurando a primeira posição livre, o que dá uma complexidade $O(n)$.

3.3 Estrutura de Inodes e Diretórios

Cada arquivo ou diretório tem exatamente um inode. Nele guardamos o tipo (arquivo ou diretório), o tamanho em bytes, um contador de referências (`ref_count`), oito ponteiros diretos para blocos de dados e, como extensão de bônus, um ponteiro para bloco indireto e dois timestamps. Diretórios, por sua vez, são tratados como arquivos especiais cujo conteúdo é uma sequência de entradas de diretório (`dirent_t`), cada uma associando um nome de até 31 caracteres a um número de inode.

Depois que `fs_init()` formata o disco, a estrutura mínima pedida pelo enunciado já existe automaticamente: o inode raiz ("/") e os três subdiretórios exigidos, `/home`, `/tmp` e `/bin`, prontos para os testes de navegação.

3.4 Resolução de Caminhos (`path_lookup`)

A função `path_lookup()` transforma um caminho absoluto, como `"/home/aluno/notas.txt"`, no inode correspondente. Ela sempre começa pelo inode raiz e vai consumindo o caminho um componente de cada vez, com a ajuda da função `next_component()`, procurando em cada etapa a entrada de diretório com aquele nome (`find_entry()`) e avançando para o inode encontrado. Se algum componente do meio do caminho não for um diretório, ou simplesmente não existir, a função retorna nulo.

Essa função acaba sendo a base de quase todas as outras operações do sistema: `mkdir()`, `create()` e `unlink()` usam uma variação dela, a `split_parent()`, para achar o diretório pai de um caminho antes de criar ou remover uma entrada.

3.5 Criação de Diretórios e Arquivos

`mkdir()` e `create()` seguem o mesmo fluxo: acham o diretório pai com `split_parent()`, conferem se já existe uma entrada com o mesmo nome para evitar

duplicidade, alocam um novo inode, inicializam os metadados dele (tipo, tamanho zero, permissões e timestamps padrão) e, por fim, registram a nova entrada no diretório pai.

Uma decisão que tomamos foi como definir o descritor de arquivo (fd). Como o enunciado não pede uma função `open()` e as assinaturas de `read()/write()` já recebem um fd pronto, decidimos que `create()` devolveria diretamente o número do inode alocado, e esse número passaria a ser usado como fd. Isso evita ter que manter uma tabela de arquivos abertos e mantém a implementação dentro do escopo educacional do projeto.

3.6 Escrita e Leitura de Arquivos

`write()` calcula quantos blocos são necessários a partir do tamanho pedido, aloca ou reaproveita esses blocos e copia os dados do buffer do usuário para a área de dados do disco virtual, atualizando o campo `size` do inode no final. `read()` faz o caminho inverso: percorre os blocos do inode e reconstrói o conteúdo no buffer que foi passado, sem ultrapassar o tamanho real do arquivo.

No início, as duas funções só trabalhavam com os oito blocos diretos do inode, o que limitava os arquivos a 8 vezes 512, ou seja, 4096 bytes. Resolvemos essa limitação com o bônus de blocos indiretos, explicado na Seção 3.8.3.

3.7 Remoção de Arquivos e Reutilização de Recursos

`unlink()` acha a entrada de diretório do caminho informado e a remove na hora. Depois, se não sobrar nenhum outro nome (link) apontando para aquele inode, o que só é possível verificar graças ao bônus de links físicos da Seção 3.8.2, os blocos de dados voltam para o `block_bitmap` e o inode volta para o `inode_bitmap`, ficando disponíveis para o próximo `create()` ou `mkdir()`.

Esse reaproveitamento é justamente o que o Cenário 8 dos testes valida, na Seção 5.8: depois de remover um arquivo, criar um novo arquivo reaproveita exatamente o mesmo número de inode e o mesmo bloco de dados que tinham acabado de ser liberados.

3.8 Funcionalidades Extras (Bônus)

Além do que era obrigatório, implementamos quatro das funcionalidades extras sugeridas pelo enunciado. Elas estão detalhadas a seguir.

3.8.1 Timestamps

Cada inode passou a guardar dois timestamps: `created_at`, definido na hora da criação, e `modified_at`, atualizado toda vez que um `write()` é bem-sucedido. Como a placa virt do QEMU que usamos não tem um relógio de tempo real acessível, usamos o contador de ciclos do processador, o registrador `time` do RISC-V, lido com a instrução `csrr`. Ou seja, é um timestamp monotônico, contado a partir do início da execução, e não uma data de calendário. Foi a simplificação possível dentro do ambiente bare-metal do projeto.

3.8.2 Links Físicos (Hard Links)

Implementamos `link(oldpath, newpath)`, que cria uma nova entrada de diretório apontando para o inode de um arquivo que já existe e aumenta o seu contador de referências (`ref_count`), e `open(path)`, que recupera o descritor (`fd`) de um arquivo já existente a partir do caminho dele. Também precisamos adaptar `unlink()` para diminuir o `ref_count` a cada remoção e só liberar de fato os blocos e o inode quando esse contador chega a zero. Assim reproduzimos a mesma semântica de hard link do Unix: o conteúdo de um arquivo só desaparece de verdade quando o último nome que aponta para ele é removido.

3.8.3 Blocos Indiretos

Para passar do limite de 4096 bytes imposto pelos oito blocos diretos, adicionamos ao inode um campo `indirect`: um bloco de dados que passa a funcionar como uma tabela com até 128 ponteiros extras, já que 512 bytes divididos por 4 bytes de cada ponteiro dá esse número. As funções internas `block_for_write()` e `block_for_read()` traduzem o índice lógico de um bloco do arquivo para o bloco físico correspondente, e só alocam o bloco indireto quando o arquivo realmente ultrapassa os oito blocos diretos. Com essa mudança, um arquivo passou a poder ter até $(8 + 128)$ vezes 512 bytes, ou seja, 69632 bytes, quase 68 KiB.

3.8.4 Permissões

Adicionamos ao inode um campo `perm`, com permissões simplificadas de leitura, escrita e execução (`rwX`) em um único conjunto de 3 bits, sem separar usuário, grupo e outros, já que não implementamos suporte a múltiplos usuários (ver Seção 3.8.5). Por padrão, arquivos novos nascem com `rw-` e diretórios com `rwX`. A função `chmod(path, mode)` permite trocar essas permissões, que passaram a ser conferidas

logo no início de `read()` e `write()`: se o bit necessário não estiver marcado, a operação já retorna erro (-1) sem tocar em nada.

3.8.5 Funcionalidades Bônus Não Implementadas

Por causa do prazo e depois de pesar o custo-benefício, decidimos não implementar três itens da lista de bônus:

- Links simbólicos. Precisariam de um novo tipo de inode (symlink) cujo dado seria o caminho de destino, além de mudar `path_lookup()` para reconhecer e seguir esse tipo de inode durante a resolução. Não chegamos a implementar isso nesta entrega.
- Cache de blocos. Como o nosso disco virtual já é inteiramente RAM, alocado via `kmalloc()`, um cache de blocos não traria nenhum ganho de desempenho real neste projeto, seria só mais uma camada de indireção sem benefício prático. Por isso, decidimos deixá-lo de fora de propósito.
- Suporte a múltiplos usuários. Isso exigiria um conceito de usuário ou dono integrado ao resto do kernel, tasks e escalonador incluídos, e hoje o kernel não tem noção nenhuma de usuário. Implementar esse bônus isolado, só dentro do TreeFS, sem essa integração, seria só decoração.

Esses três pontos ficam registrados como sugestões naturais de evolução do projeto.

4. Códigos Importantes da Implementação

4.1 Resolução de Caminhos

A função a seguir percorre a árvore de diretórios componente por componente até localizar o inode final:

```
inode_t *path_lookup(const char *path) {
    if (!path || path[0] != '/') return 0;
    uint32_t current = g_root_ino;
    if (path[1] == '\0') return &inode_table[current];

    const char *p = path;
    char comp[MAX_FILENAME];
    while (next_component(&p, comp)) {
        if (inode_table[current].type != TYPE_DIR) return 0;
        int found = find_entry(current, comp);
        if (found < 0) return 0;
        current = (uint32_t)found;
    }
    return &inode_table[current];
}
```

4.2 Tradução de Bloco Lógico para Físico (Blocos Indiretos)

Trecho responsável por localizar/alocar o bloco físico correspondente ao índice lógico de um arquivo, incluindo a indireção quando o índice ultrapassa os oito blocos diretos:

```
static int block_for_write(inode_t *inode, uint32_t index) {
    if (index < DIRECT_BLOCKS) {
        if (inode->blocks[index] == 0) {
            int blk = block_alloc();
            if (blk < 0) return -1;
            inode->blocks[index] = (uint32_t)blk;
        }
        return (int)inode->blocks[index];
    }
    uint32_t ind_index = index - DIRECT_BLOCKS;
    if (ind_index >= PTRS_PER_BLOCK) return -1;
    if (inode->indirect == 0) {
        int blk = block_alloc();
        if (blk < 0) return -1;
        inode->indirect = (uint32_t)blk;
        memset(data_blocks + ((uint32_t)blk * BLOCK_SIZE), 0, BLOCK_SIZE);
    }
    uint32_t *ptrs = (uint32_t *) (data_blocks + (inode->indirect * BLOCK_SIZE));
    if (ptrs[ind_index] == 0) {
        int blk = block_alloc();
        if (blk < 0) return -1;
        ptrs[ind_index] = (uint32_t)blk;
    }
    return (int)ptrs[ind_index];
}
```

4.3 Remoção com Contagem de Referências (Links Físicos)

Trecho de unlink() responsável por só liberar blocos e inode quando não sobra nenhum outro nome apontando para o arquivo:

```
remove_dir_entry(parent_ino, name);

if (inode->ref_count > 0) inode->ref_count--;
if (inode->ref_count > 0) return 0; // ainda ha outros links

// ultimo link: libera blocos diretos, indiretos e o inode
...
```

5. Testes e Resultados Obtidos

Os testes foram implementados diretamente em kernel_main() (arquivo main.c) e são executados automaticamente a cada boot do kernel, sem qualquer interação manual, imprimindo cada etapa via UART. As subseções a seguir reproduzem os oito cenários obrigatórios exigidos pelo enunciado (Seções 5.1 a 5.8) e os quatro testes das funcionalidades bônus (Seções 5.9 a 5.12). Cada uma indica onde inserir o print de tela correspondente, capturado a partir da execução no ambiente Codespaces.

5.1 Cenário 1 – Listagem da Raiz

Objetivo: verificar que, após fs_init(), a estrutura mínima obrigatória (/home, /tmp, /bin) existe e é corretamente listada por ls("/").

Saída esperada (trecho do console UART):

```
[TreeFS] Formatando o disco virtual...
[TreeFS] Sistema de arquivos inicializado com sucesso!

[Cenário 1] Listagem da raiz -> ls("/")
home
tmp
bin
```

```
=== VALIDACAO DO TREEFS ===

[TreeFS] Formatando o disco virtual...
[TreeFS] Sistema de arquivos inicializado com sucesso!

[Cenário 1] Listagem da raiz -> ls("/")
home
tmp
bin
```

Figura 1. Saída de `ls("/")` logo após a formatação do disco virtual.

5.2 Cenário 2 – Criação de Diretório

Objetivo: validar `mkdir("/home/aluno")`, confirmando que o novo diretório passa a existir e aparece na listagem de `/home`.

Saída esperada (trecho do console UART):

```
[Cenário 2] Criacao de diretorio -> mkdir("/home/aluno")
aluno
```

```
[Cenário 2] Criacao de diretorio -> mkdir("/home/aluno")
aluno
```

Figura 2. Diretório “aluno” criado dentro de `/home`.

5.3 Cenário 3 – Criação de Arquivo

Objetivo: validar `create("/home/aluno/notas.txt")`, confirmando que um novo inode é alocado e seu número (usado como descritor de arquivo) é devolvido pela função.

Saída esperada (trecho do console UART):

```
[Cenário 3] Criacao de arquivo -> create("/home/aluno/notas.txt")
fd (inode) retornado: 5
```

```
[Cenário 3] Criacao de arquivo -> create("/home/aluno/notas.txt")
fd (inode) retornado: 5
```

Figura 3. Arquivo `notas.txt` criado; `fd` devolvido corresponde ao inode alocado.

5.4 Cenário 4 – Escrita

Objetivo: validar `write(fd, "Sistemas Operacionais", 22)`, confirmando que a operação é concluída sem erros e que o tamanho do arquivo é atualizado.

Saída esperada (trecho do console UART):

```
[Cenário 4] Escrita -> write(fd, "Sistemas Operacionais", 22)
Escrita concluída.
```

```
[Cenário 4] Escrita -> write(fd, "Sistemas Operacionais", 22)
Escrita concluída.
```

Figura 4. Escrita de 22 bytes no arquivo *notas.txt*.

5.5 Cenário 5 – Leitura

Objetivo: validar `read(fd, buffer, 22)`, confirmando que o conteúdo lido corresponde exatamente ao que foi escrito no cenário anterior.

Saída esperada (trecho do console UART):

```
[Cenário 5] Leitura -> read(fd, buffer, 22)
Conteúdo lido: Sistemas Operacionais
```

```
[Cenário 5] Leitura -> read(fd, buffer, 22)
Conteúdo lido: Sistemas Operacionais
```

Figura 5. Leitura do conteúdo gravado, íntegro.

5.6 Cenário 6 – Remoção

Objetivo: validar `unlink("/home/aluno/notas.txt")`, confirmando que o arquivo deixa de existir e não aparece mais na listagem do diretório.

Saída esperada (trecho do console UART):

```
[Cenário 6] Remocao -> unlink("/home/aluno/notas.txt")
ls("/home/aluno") apos remocao:
(lista vazia)
```

```
[Cenário 6] Remocao -> unlink("/home/aluno/notas.txt")
ls("/home/aluno") apos remocao:
```

Figura 6. Diretório */home/aluno* vazio após a remoção do arquivo.

5.7 Cenário 7 – Navegação Hierárquica

Objetivo: validar a navegação em múltiplos níveis da árvore com `ls("/home")`, confirmando que o diretório aluno criado no Cenário 2 continua acessível.

Saída esperada (trecho do console UART):

```
[Cenário 7] Navegacao hierarquica -> ls("/home")
aluno
```

```
[Cenário 7] Navegacao hierarquica -> ls("/home")
aluno
```

Figura 7. Navegação hierárquica confirmando o diretório *aluno*.

5.8 Cenário 8 – Reutilização de Inodes e Blocos

Objetivo: criar um novo arquivo após a remoção do Cenário 6 e confirmar que o mesmo número de inode e o mesmo bloco de dados liberados são reaproveitados, validando o funcionamento correto dos bitmaps de controle.

Saída esperada (trecho do console UART):

```
[Cenario 8] Reutilizacao de inodes/blocos liberados
Novo fd apos remocao anterior (deve reaproveitar o inode liberado): 5
Bloco de dados reaproveitado: 5
```

```
[Cenario 8] Reutilizacao de inodes/blocos liberados
Novo fd apos remocao anterior (deve reaproveitar o inode liberado): 5
Bloco de dados reaproveitado: 5
=====
```

Figura 8. Reutilização do inode 5 e do bloco 5, previamente liberados.

5.9 Teste Bônus 1 – Timestamps

Objetivo: confirmar que `created_at` é definido na criação e permanece fixo, enquanto `modified_at` muda a cada `write()` subsequente.

Saída esperada (trecho do console UART):

```
[Bonus 1] Timestamps (created_at / modified_at)
Logo apos create(): criado (tick): 568480 | modificado (tick): 568480
Apos write(): criado (tick): 568480 | modificado (tick): 574390
```

```
=== FUNCIONALIDADES EXTRAS (BONUS) ===

[Bonus 1] Timestamps (created_at / modified_at)
Logo apos create():
[stat] /tmp/relatorio.txt
tipo: arquivo
tamanho: 0 bytes
links (ref_count): 1
permissoes: rw-
criado (tick): 479390
modificado (tick): 479390
Apos write() (modified_at deve mudar):
[stat] /tmp/relatorio.txt
tipo: arquivo
tamanho: 2 bytes
links (ref_count): 1
permissoes: rw-
criado (tick): 479390
modificado (tick): 484750
```

Figura 9. `created_at` inalterado e `modified_at` atualizado após a escrita.

5.10 Teste Bônus 2 – Links Físicos

Objetivo: criar um segundo nome (link) para o mesmo arquivo, confirmar `ref_count = 2`, remover um dos nomes e confirmar que o conteúdo continua acessível pelo nome restante; só após o segundo `unlink()` o inode deve ser liberado.

Saída esperada (trecho do console UART):

```
[Bonus 2] Links fisicos (hard links)
ref_count apos link(a.txt, b.txt): 2
unlink(a.txt): conteudo ainda acessivel via b.txt -> "compartilhado"
unlink(b.txt): agora libera o inode de fato (era o ultimo link)
```

```
[Bonus 2] Links fisicos (hard links)
Apos link(a.txt, b.txt), ls("/home/aluno"):
outro.txt
a.txt
b.txt
Metadados de a.txt (ref_count deve ser 2):
[stat] /home/aluno/a.txt
  tipo: arquivo
  tamanho: 13 bytes
  links (ref_count): 2
  permissoes: rw-
  criado (tick): 488890
  modificado (tick): 489010
unlink(a.txt): conteudo deve continuar acessivel via b.txt
Conteudo lido via b.txt: compartilhado
[stat] /home/aluno/b.txt
  tipo: arquivo
  tamanho: 13 bytes
  links (ref_count): 1
  permissoes: rw-
  criado (tick): 488890
  modificado (tick): 489010
unlink(b.txt): agora libera o inode de fato (era o ultimo link)
```

Figura 10. *ref_count = 2 após o link e conteúdo preservado após o primeiro unlink().*

5.11 Teste Bônus 3 – Blocos Indiretos

Objetivo: escrever e ler um arquivo de 5000 bytes (acima dos 4096 bytes suportados apenas por blocos diretos) e verificar byte a byte a integridade do conteúdo recuperado.

Saída esperada (trecho do console UART):

```
[Bonus 3] Blocos indiretos (arquivo maior que 8 blocos diretos)
Arquivo de 5000 bytes (8 blocos diretos + indiretos): OK, conteudo integro
```

```
[Bonus 3] Blocos indiretos (arquivo maior que 8 blocos diretos)
Arquivo de 5000 bytes (8 blocos diretos + indiretos): OK, conteudo integro
```

Figura 11. *Arquivo de 5000 bytes gravado e lido corretamente via blocos diretos + indiretos.*

5.12 Teste Bônus 4 – Permissões

Objetivo: remover a permissão de escrita de um arquivo (chmod para r--) e confirmar que write() passa a ser bloqueado enquanto read() continua funcionando; em seguida, restaurar a permissão e confirmar que write() volta a funcionar.

Saída esperada (trecho do console UART):

```
[Bonus 4] Permissoes (rwx simplificado por inode)
chmod("/tmp/protegido.txt", PERM_READ): agora e somente leitura
write() sem permissao: BLOQUEADO (retornou -1), como esperado
read() continua funcionando (permissao de leitura mantida): dado inicial
write() apos restaurar permissao: sucesso
```

```
[Bonus 4] Permissões (rwx simplificado por inode)
[stat] /tmp/protegido.txt
tipo: arquivo
tamanho: 12 bytes
links (ref_count): 1
permissões: rw-
criado (tick): 512480
modificado (tick): 512590
chmod("/tmp/protegido.txt", PERM_READ): agora e somente leitura
[stat] /tmp/protegido.txt
tipo: arquivo
tamanho: 12 bytes
links (ref_count): 1
permissões: r--
criado (tick): 512480
modificado (tick): 512590
write() sem permissão: BLOQUEADO (retornou -1), como esperado
read() continua funcionando (permissão de leitura mantida): dado inicial
chmod("/tmp/protegido.txt", PERM_READ|PERM_WRITE): restaura escrita
write() após restaurar permissão: sucesso
=====
```

Figura 12. *write()* bloqueado com permissão *r--* e restaurado após novo *chmod()*.

5.13 Resumo dos Resultados

A tabela a seguir resume todos os cenários testados e o status observado durante a execução no QEMU:

#	Cenário	Função(ões) testada(s)	Resultado esperado	Status
1	Listagem da raiz	ls	home, tmp, bin	OK
2	Criação de diretório	mkdir	aluno listado em /home	OK
3	Criação de arquivo	create	fd (inode) retornado	OK
4	Escrita	write	22 bytes gravados	OK
5	Leitura	read	conteúdo íntegro	OK
6	Remoção	unlink	entrada removida	OK
7	Navegação hierárquica	ls	aluno listado	OK
8	Reuso de inode/bloco	create + unlink	mesmo fd/bloco reaproveitado	OK
B1	Timestamps	fs_stat	modified_at atualizado	OK
B2	Links físicos	link, open, unlink	ref_count = 2; conteúdo preservado	OK
B3	Blocos indiretos	write, read	5000 B íntegros	OK
B4	Permissões	chmod	write bloqueado/restaurado	OK

6. Análise e Discussão

6.1 Dificuldades Encontradas

- Dimensionamento do heap. O disco virtual, somando bitmaps, tabela de inodes e área de dados, precisa de cerca de 137 KiB contíguos, um valor bem maior do que o heap de 64 KiB que tínhamos herdado da avaliação M2. Tivemos que aumentar o heap para 256 KiB em *memory.c*, e essa decisão fez sentido porque a placa virt do QEMU está configurada com 128 MiB de RAM (parâmetro *-m 128M* do *Makefile*), então sobrava espaço de sobra para crescer.
- Ausência de *open()*. Como o enunciado não define uma função de abertura de arquivo, tivemos que decidir sozinhos como um fd seria obtido para as chamadas seguintes de *read()*/*write()*. A solução que escolhemos foi fazer o

fd ser igual ao número do inode e devolvê-lo direto por create(). Isso simplificou bastante a implementação sem violar nenhuma assinatura exigida, mas nos obrigou a criar uma função open() própria, de bônus, para reabrir arquivos já existentes durante os testes de links físicos.

- Limite de um único bloco por diretório. Por simplicidade, cada diretório usa só o seu primeiro bloco de dados, de 512 bytes, para guardar entradas, o que dá para cerca de 14 entradas por diretório. Essa limitação não atrapalhou os testes que fizemos, mas seria o primeiro ponto a melhorar se o sistema precisasse suportar diretórios com muito mais arquivos.
- Integração entre os bônus. Juntar links físicos com blocos indiretos pediu bastante atenção dentro de unlink(), porque ela precisa liberar tanto os blocos diretos quanto os apontados pelo bloco indireto, e só quando o ref_count chega a zero. Se essa ordem de condições fosse implementada errado, o sistema vazaria blocos ou liberaria dados que ainda estavam sendo referenciados por outro nome.

6.2 Resultados Obtidos

Todos os oito cenários obrigatórios de validação pedidos pelo enunciado, além dos quatro testes das funcionalidades bônus, rodaram com sucesso no ambiente de simulação (QEMU, máquina virt, RISC-V 64 bits), como mostra a Seção 5. O sistema cria, navega, armazena, lê e remove arquivos corretamente usando só caminhos absolutos, e a reutilização de inodes e blocos mostra que os bitmaps de controle acompanham fielmente o estado do disco virtual ao longo de várias operações seguidas.

7. Conclusão

Fazer o TreeFS nos deu uma noção bem mais prática de como funcionam por dentro os sistemas de arquivos hierárquicos baseados em inodes, principalmente a separação entre metadados e dados, o papel dos bitmaps no controle de espaço livre e a travessia recursiva da árvore de diretórios durante a resolução de caminhos.

Integrar tudo isso com o que já tínhamos da M2, o gerenciador de memória e o escalonador preemptivo, também deixou claro o quanto um alocador dinâmico confiável é importante: o disco virtual inteiro do TreeFS depende diretamente do kmmalloc() estar funcionando direito, o que mostra bem como os subsistemas de um kernel dependem uns dos outros.

Por fim, implementar as quatro funcionalidades bônus, timestamps, links físicos, blocos indiretos e permissões, deu para explorar, mesmo que de um jeito simplificado, mecanismos que existem em sistemas de arquivos reais. Ficou claro para nós que decisões de projeto que parecem pequenas, como a semântica de um contador de referências ou a forma de traduzir índices lógicos para blocos físicos, acabam tendo um impacto direto na corretude e na robustez do sistema inteiro.